IMPERIAL

MEng Individual Project

Imperial College London

Department of Computing

Tools for Foundational Verification of Probabilistic Programs

Author:
Edwin Fernando

Supervisor:
Shing Hin Ho
Dr. Azalea Raad

Second Marker:
Prof. Nicolas Wu

May 22, 2026

Contents

Chapter 1

Introduction

Probabilistic programming languages (Hakaru, Stan, Gen, etc Narayanan et al. [2016]; Carpenter et al. [2017]; Cusumano-Towner et al. [2019]) offer a principled way of creating statistical models complex probability distributions over multiple random variables, as code. The programmer (statistician) can then interact with the model, evaluating (sampling from) the generated distribution. This decouples the specification of a model from the inference algorithm used for sampling. Probabilistic programming is in fact also pervasive in computer science, though generally not cleanly abstracted into a DSL. Examples include Zero Knowledge Proof Systems (Blockchain) Verified-zkEVM [2025a], Differential Privacy and Machine Learning (TensorFlow Probability Dillon et al. [2017]). Formal reasoning about programs in these critical applications may be highly beneficial.

The aim of this project is to extend the reach of foundational verification in this direction. The main contributions are `C1`. `C2` are tentative ideas, as I am trying to expand the scope of this project. I believe it is possible to make a contribution beyond formalisation of existing work, because of the progress in `C1`.

- **C1**: Implementing (formalising) Lilac Li et al. [2023] and BaSL Ho et al. [2025] separation logics in the Lean theorem prover, with "tactical support", using Iris Jung et al. [2018].
- **C2.1**: Develop Lilac + higher order functions - continuous distributions: a separation logic for Higher order discrete probabilistic programs (without Bayesian update)
- **C2.2**: Instantiate the loom framework of multi-modal foundational verification Gladshtein et al. [2026] for probabilistic separation logic
- **C2.3**: Compositional Symbolic execution using Soteira Ayoun et al. [2025] for probabilistic separation logics

C1 is one of the first (if not the first) foundationally verified (in Lean) implementation of a probabilistic separation logic (PSL). It would offer the ability to foundationally verify the programs (such as examples given in Li et al. [2023]; Ho et al. [2025]) using tactics in the style of Iris. There is in fact a simultaneous project formalising the Bluebell PSL Verified-zkEVM [2025b] for the purpose of verifying the core of a Zero Knowledge Proof system Verified-zkEVM [2025a].

C2.1 would be a novel theoretical contribution, exploring the possibility to explore higher order functions, in the simplified discrete setting. In the continuous setting there are serious complications (namely the category of measurable spaces in which these programs are interpreted is not cartesian closed), which would be avoided.

C2.2: Is very interesting but potentially intractable, because Loom doesn't yet support separation logic

C2.3 Is an interesting theoretical question, but is possible not well motivated practically

My progress on the Lilac formalisation is on GitHub.

Chapter 2

Background

Probabilistic programming is a paradigm where we are defining statistical models using code. It is a means of describing complex distributions over some space in a structured way. Probabilistic programming language (PPL) also come with an interpreter (probabilistic inference engine) which allows the programmer to sample a term from the distribution denoted by a program and indeed sample distributions while constructing new distributions.

More interestingly a Bayesian probabilistic programming languages (BPPL) also allows one to "observe" an event and update their belief of the world (update the probability distribution they associate with some ocurrence). Here we take the bayesian perspective underpinned by Baye's law which describes how we update a probability distribution based

A simple example taken from Staton [2017] is reproduced below to show how

2.1 bayesian updates

This section can be skipped if the comparison between PPLs which support vs do not support the observe construct is not of interest. The logic being formalised in this project does not support the observe construct.

`<insert: example of bus frequency on weekends vs weekdays>`

`<insert: State Baye's law through the example>`

2.2 Measure Theory

In order to understand the semantics of a probabilistic programming, a formal definition for probability is needed. Measure theory underpins probability theory and an overview is given here. We want to be able to talk about the probabilities of some event occurring, which is modelled by a `random variable`. In order to define a random variable we first need to define Measures and Measurable Space. A measurable space is a pair (Ω, Σ_Ω), where Ω is some Set, and $\Sigma_\Omega \subseteq \mathcal{P}(\Omega)$ is called the σ-algebra. We interpret each element of Σ_Ω to be an event (such as a coin flip landing heads). We would like the σ-algebra to be closed under complements and countable unions. From the presentation so far, it may seem like all singleton sets must be events but due to technical reasons (see Axler [2020] Ch. 2), this cannot be in certain cases where Σ is infinite.

A measure on a measurable space is a function $\mu : \Sigma_\Omega \to [0, \infty]$.

A probability measure on the other hand is more restrictive: $\mu : \Sigma_\Omega \to [0, 1]$. It gives the probability of an event $E \in \Sigma\_\Omega$ ocurring. A similar but more general notion than what a measure gives us, is a random variable: a measurable function between two measurable spaces. I say "more general notion", since using the identity measurable function yields the measure we started with. A measurable function denoted $f : (\Omega_1, \Sigma_\Omega) \xrightarrow{m} (\Omega_2, \mathcal{G})$ $f : \Omega_1 \xrightarrow{m} \Omega_2$ is well defined when there is an ambient measurable space structure $(\Omega_1, \Sigma_\Omega)$ and $(\Omega_2, \mathcal{G})$. f allows us to talk about the distribution of a particular attribute of in the event space. For example $\Omega_1 = \{1, \ldots, 6\}^2$ may

be the set of all events associated with playing 2 dice, while f may map to the sum of the two dice, $\Omega_2 = \{1, \ldots, 12\}$. A measurable function f, transforms any measure μ on $(\Omega_1, \Sigma_\Omega)$ to the pushforward measure μ' on $(\Omega_2, \mathcal{G})$. In order for this to always be well defined we require the following measurability to be satisfied by all measurable functions: $\forall G \in \mathcal{G}, f^{-1}(G) \in \Sigma_\Omega$.

A Lebesgue integral is a generalisation of the standard (Reimann) integral. Intuitively the lebesgue and reimann integral will coincide on the spaces such as $\mathbb{R}$, where the domain of integration is continuous. The lebesgue integral relaxes that requirement. The lesbesgue integral of a measurable function $f : \Omega \to [0, \infty]$ with respect to a measure $\mu : \Sigma_\Omega \to [0, \infty]$ is a number defined by:

$$\int_\Omega f \, d\mu := \sup \left\{ \sum_{i=1}^{n} f(\omega_i)\, \mu(U_i) \;\middle|\; \{U_i \in \Sigma_\Omega\}_{i=1}^{n} \text{ disjoint cover of } \Omega \right\}.$$

To get an intuition remember that integrals are supposed to add the values of function by breaking the domain up into infinitesimal pieces. Observe that the integral is a weighted sum of $f(\omega)$, weighted by the measure. The integral is iterating over the entire space by picking some choice of measurable disjoint sets whose union is the Ω (disjoint cover of Ω). And for each measurable set (U_i) it picks a representative point ω evaluates f on ω and weights it by how large the set is (μU_i). The supremum and infimum together enforce that the chosen disjoint cover is chosen as finely (small measure) as possible.

2.2.1 Borel spaces and lebesgue measures

2.2.2 Product of measurable spaces

2.3 Semantics of Probabilistic Programs

We first present APPL [Li et al., 2023, appendix A], a probabilistic programming language without bayesian updates. Syntactically it looks like a basic WHILE language. In Appl, there isn't an explicit `sample` construct, sampling is realised in the semantics of sequencing as explained later. Though the language is first order, PPLs still have a non-trivial, novel denotational semantics. The central difference is that types are interpreted as `MeasurableSpace` rather than just `Set` from which everything else follows. Concretely $[\![A]\!] = (\mathbb{A}, \Sigma_{\mathbb{A}})$. In our PPL we would like to express measures and manipulate measures. The type constructor GA is interpreted as the (meta) type $ProbMeasure[\![A]\!]$, the ~~`Set`~~ `MeasurableSpace` of measures over $[\![A]\!]$. So in order for GA to be a valid (object) type, there must be some canonical MeasurableSpace $\Sigma_{\mathrm{G}A} \subseteq \mathcal{P}(\Sigma_A \to [0, 1])$. This is indeed true, but we don't reprove the mathematical foundations on which the semantics stands, in this report.

`<insert: Why measurableSpaces?>`

$$\begin{aligned}
A, B &::= A \times B \mid \mathit{bool} \mid \mathit{real} \mid A^n \mid \mathit{index} \mid \mathrm{G}A \\
p &\in [0, 1] \\
r &\in \mathbb{R} \\
n &\in \mathbb{N} \\
\oplus &\in \text{Arith} := \{+, -, \times, /, \hat{}\} \\
< &\in \text{Cmp} := \{<, \leq, =\} \\
M, N, O &::= X \mid \text{ret } M \mid X \leftarrow M;\ N \mid \\
&\quad (M, N) \mid \text{fst } M \mid \text{snd } M \mid \\
&\quad \mathrm{T} \mid \mathrm{F} \mid \text{if } M \text{ then } N \text{ else } O \mid \text{flip } p \mid \\
&\quad r \mid M \oplus N \mid M < N \mid \text{unif } [0, 1] \mid \\
&\quad [M, \ldots, M] \mid [M|N] \mid \text{for}(n, M_{\text{init}}, i\ X.\, M_{\text{step}})
\end{aligned}$$

Figure 2.1: Syntax of the language

$$
\begin{aligned}
[\![A \times B]\!] &= [\![A]\!] \otimes [\![B]\!] \\
[\![\textbf{bool}]\!] &= (\{T, F\}, \mathcal{P}(\{T, F\})) \\
[\![\textbf{real}]\!] &= (\mathbb{R}, \mathcal{B}(\mathbb{R})) \\
[\![A^n]\!] &= \underbrace{[\![A]\!] \otimes \cdots \otimes [\![A]\!]}_{n \text{ times}} \\
[\![\textbf{index}]\!] &= (\mathbb{N}, \mathcal{P}(\mathbb{N})) \\
[\![\textbf{GA}]\!] &= \mathcal{G}[\![A]\!] \\
[\![\textbf{1}]\!] &= \text{the one-point space}
\end{aligned}
$$

Figure 2.2: Denotational semantics of types

$$
\begin{aligned}
[\![X]\!]\rho &= \rho(X) \\
[\![\text{ret } M]\!]\rho &= \text{ret } [\![M]\!]\rho \\
[\![X \leftarrow M;\ N]\!]\rho &= v \leftarrow [\![M]\!]\rho;\ [\![N]\!][X \mapsto v] \\
[\![(M, N)]\!]\rho &= ([\![M]\!]\rho,\ [\![N]\!]\rho) \\
[\![\text{fst } M]\!]\rho &= \pi_1([\![M]\!]\rho) \\
[\![\text{snd } M]\!]\rho &= \pi_2([\![M]\!]\rho) \\
[\![\text{T}]\!]\rho &= \text{T} \\
[\![\text{F}]\!]\rho &= \text{F} \\
[\![\text{if } M \text{ then } N \text{ else } O]\!]\rho &= \begin{cases} [\![N]\!]\rho, & [\![M]\!]\rho = \text{T} \\ [\![O]\!]\rho, & [\![M]\!]\rho = \text{F} \end{cases} \\
[\![\text{flip } p]\!]\rho &= \text{Ber}\, p \\
[\![r]\!]\rho &= r \\
[\![M \oplus N]\!]\rho &= ([\![M]\!]\rho) \oplus ([\![N]\!]\rho) \\
[\![\text{unif } [0,1]]\!]\rho &= \text{Unif}\, [0,1] \\
[\![[M_1, \ldots, M_n]]\!]\rho &= ([\![M_1]\!]\rho, \ldots, [\![M_n]\!]\rho) \\
[\![[M[N] : A]]\!]\rho &= \begin{cases} \pi_N\big([\![M]\!]\rho\big), & 1 \le [\![N]\!]\rho \le n \\ \text{arbitrary}(A), & \text{otherwise} \end{cases} \\
[\![\text{for}(n, M_i, i\ X.\, M_s)]\!]\rho &= \text{loop}(1,\ [\![M_i]\!]\rho,\ \lambda k.\ [\![M_s]\!]\rho[i \mapsto k,\ X \mapsto v]) \\
\text{where loop}(k, u, f) &= \begin{cases} \text{ret } u, & k > n \\ v' \leftarrow f(k, v);\ \text{loop}(k+1, v', f), & \text{otherwise} \end{cases}
\end{aligned}
$$

Figure 2.3: Denotational semantics of terms

2.3.1 Types in BPPL

The setting is the category of Measurable Spaces, `Meas`. All types in our language is interpreted as an object in this category. Syntactically:

$$\mathbb{A}, \mathbb{B} ::= \mathbb{R} \mid \mathcal{P}(\mathbb{A}) \mid 1 \mid \mathbb{A} \times \mathbb{B} \mid \cdots$$

Objects in this category are of the form (A, Σ_A). Morphisms are measurable functions. The interesting type here is $\mathcal{P}(\mathbb{A})$. It's the type of probability distributions over $\mathbb{A}$. Actually not quite, since we allow any measure $\mu : \Sigma_A \to [0, \infty]$, rather than $\Sigma_A \to [0, 1]$. Now in order to be an object in the category, $\mathcal{P}(\mathbb{A})$, the set of all measures on $\mathbb{A}$, must itself form a measurable space. There is a construction to form a σ-algebra for $\mathcal{P}(\mathbb{A})$ satisfying all the requirements (we don't go into it here).

A natural next question is how these measures can be created and manipulated. The semantics of PPLs are given in a monadic framework (in other words probabilistic programs can be thought of as effectful computations). In practice this means that there is are `ret` and `bind` type constructors to construct terms of (object) type $mathrmG?$, such that

$$[\![\text{ret}]\!] : A \to \mathcal{G}A \qquad [\![\text{bind}]\!] : \mathcal{G}A \to (A \to \mathcal{G}B) \to \mathcal{G}B$$

The core intuition of how probabilistic programs work is gained by studying the semantics of let bindings **??**. But first let's explain how variables work in Appl. A standard technique for representing variables is to use a variable context. A map from this context to the denotation of an AST would be considered a program. However, in Appl, the context is a measurable space, and the map is also measurable. Precisely the context and interpretation of well-typedness in context is given as:

$$[\![\Gamma, X : A]\!] = [\![\Gamma]\!] \otimes [\![A]\!] \qquad [\![M]\!] : [\![\Gamma]\!] \xrightarrow{\text{m}} [\![A]\!]$$

2.3.2 The let binding problem

Staton provided a compositional semantics for first-order Bayesian probabilistic programs in Staton [2017], which forms the foundation of program logics, validation of inference algorithms, etc. The scope of this project is entirely within first-order probabilistic programs (probability distributions over function spaces are not expressible). The programming language considered in Lilac uses a simpler semantics since it doesn't support bayesian updates. So in the following, we present the relevant excerpt of the semantics.

The key contribution of the paper is the introduction of `s-finite kernels`, motivated by validating a basic commutativity property that programmers expect. Specifically: when neither x

```
let x = t in          =     let y = u in
let y = u in                let x = t in
v                           v
```

Figure 2.4: commutative program transformation

is free in u nor y is free in t. Another benefit of this semantics is a term-wise compositionality, ie, the semantics of every subterm in a program is well-defined, not just whole programs. This is just another way of saying the semantics of open programs (those with free variables), as well as closed programs is well-defined, respectively. Validating proof rules in a logic is much better formulated in terms of such a compositional semantics. s-finite kernels denote open programs and s-finite measures denote closed programs.

At this point it is important to disseminate the difference between Lilac's lack of bayesian updates (call this APPL) and the more general language with bayesian updates (call this BPPL). As justified in the 2.1, it is necessary to work with (unnormalised) measures when

2.3.3 Semantics of Let binding

?? Probabilistic programs are interpreted using the Giry monad Giry [1982] in the category of Measurable Space, `Meas`. Probability is an effect. This interpretation gives us the semantics of sequencing (let-binding) so we go through it below. Kernels also naturally show up in type of the monadic bind, which motivates it.

The Giry functor takes some object (measurable space) (A, Σ_A) to a measurable space on the measures of $(\Sigma_A \to [0, \infty], \cdots)$ (the σ-algebra is ommitted). We'll denote $\Sigma_A \to [0, \infty]$ as `Measure A`, for readability.

The explanation focuses on intuition through diagrams. For a measurabe space (A, Σ_A), a probability measure is of type $\mu : \Sigma_A \to [0, 1]$. This gives a distribution on A. The intuition behind μ is that we can sample some event using it. A kernel from A to B has type $\kappa : A \to MeasureB$. Each triangle below is a kernel. And the types of it input and output are indicated, and we compose the kernels along the type B as $\kappa \underset{B}{\circ} \iota$. The types of the induvidual kernels and composed kernel of Fig 2.3.3, is given below

$$\iota : A \to \text{Measure } B \tag{2.1}$$
$$\kappa : B \to \text{Measure } C \tag{2.2}$$
$$\kappa \underset{B}{\circ} \iota : A \to \text{Measure } C \tag{2.3}$$

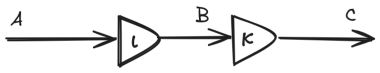

Figure 2.5: A simple probabilistic program

The interpretation of the kernel ι is that for every (deterministic) value of A it gets, it will give you a (probabilistic value) distribution over type B. So composition here, is sampling from the `Measure` B, getting a deterministic value of type B and passing that to the kernel κ. This sampling is done ranging over all the values of B in proportion to their distribution, and taking a weighted average of the values of type `Measure` C. More precisely, this is the integral: $\kappa \underset{B}{\circ} \iota = \lambda a. \int \lambda b.\ \kappa(b)\ d\iota(a)$ (integral will be explained in 2.2 in the final draft). Notice that $\circ$ has a type very similar to the monadic bind (in the Giry monad). We now write $\mathcal{G}(A)$ instead of `Measure` A, for the Giry functor. $\circ : ((B) \to \mathcal{G}(C)) \to ((A) \to \mathcal{G}(B)) \to \mathcal{G}(C)$. If we just swap the arguments and partially apply an $a : A$ on ι, we see that the bind of the Giry monad composes a measure with a kernel like so $\iota(a) \gg\!= \kappa : \mathcal{G}(B) \to (B \to \mathcal{G}(C)) \to \mathcal{G}(C)$

The most interesting part of Staton's commutative semantics is that when kernels have multiple inputs (this does extend into the realm of multi-categories but we don't go into that here). To clarify, these kernels have multiple deterministic values they take in order to return a single probabilistic value (a measure). The commutativity property illustrated by the equality of the two programs in Fig 2.3.3.

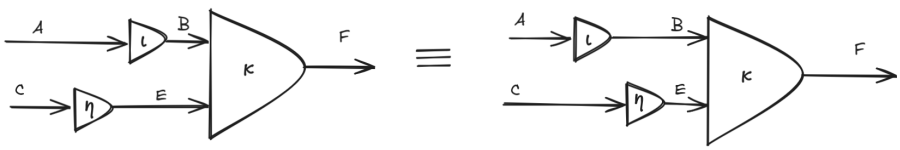

Figure 2.6: Commutativity of let bindings

$\kappa \underset{B}{\circ} \iota$ is given by the following on the left and right respectively

$\lambda a\ c. \int \lambda e.\ \left(\int \lambda b.\ \kappa(b, e)\ d\iota(a)\right) d\eta(c)$

$\lambda a\ c. \int \lambda b.\ \left(\int \lambda e.\ \kappa(b, e)\ d\eta(c)\right) d\iota(a)$

Though these look rather unreadable, all that's happening is swapping the order of integration, and there is a theorem which allows us to this in certain conditions. In Staton [2017] it is shown that this theorem holds if instead of general kernels, we restrict ourselves to s-finite-kernels. A kernel $\kappa : A \to \mathcal{G}(B)$ is finite if there is finite $r \in [0, \infty)$ such that, for all a, $\kappa(a, B) < r$. κ is s-finite if there is a sequence $\kappa_1 \ \ldots \ \kappa_n \ \ldots$ of finite kernels and $\Sigma_{i=1\ldots\infty}\ \kappa_i = \kappa$.

In short topological transformations of these diagrams corresponding to the motivating example in 2.4 do not change the semantics.

2.4 Lean and related formalisation projects

Foundational verification, consists of machine checked proofs of theorems, assuming only the axioms of mathematics. Here we are concerned with verification of software, so the theorems in question express properties of the software. The axioms of mathematics that we use is a dependent type theory. In this setting of dependent type theory, checking the correctness of proof is the same thing as checking if a proof (program) is well typed, due to the Curry-Howard Isomorphism. Further the tool we use to for machine-checked proofs is an interactive theorem prover with support for proofs using "tactics". Tactical support is just metaprogramming, trying to encapsulate reasoning principles a person would use when writing pen-and-paper proofs. There are two proof assistants that were strongly considered for the purposes of this project: Lean and Rocq, former having proofs of required theorems in measure theory and the latter having the original implementation of the Iris framework. Lean was chosen since the front-end of the Iris proof mode (MoSeL, see 2.6) had been fully ported to Lean very recently.

There has been an exciting amount of recent activity in formalisations related to this work, which has enabled the formalisation of Lilac (see, 2.5.2) in Lean to be a realistic goal. Most importantly, the formalisation of the disintegration theorem in Degenne [2025] is only available in Lean at the moment. Disintegration theorem also involved the formalisation of kernels which are actually available in several other proof assistants. Quasi-borel spaces Heunen et al. [2017] (in which higher order probabilistic programs are interpreted) have been formalised in Isabelle/HOL in Hirata et al. [2023], along with kernels. s-finite-kernels can also be found in rocq Affeldt et al. [2025].

A lot of background in Lean was required to undertake this project. For example, we use the full expressivity of inductive types and inductive families. Also an understanding of propositional vs definitional equality which is related to Proof irrelevance Avigad et al. [2024] and how that's implemented in Lean's type theory by special treatment of the lowest Universe Carneiro [2019], is very helpful.

2.4.1 Inductive families in Lean

2.4.2 Excerpt of Lean type theory bringing about proof irrelevance

2.5 Separation Logic

Separation logic has been extremely productive in the last two decades in reasoning about imperative programming languages, after first being introduced in O'Hearn and Pym [1999]. The classical separation logic introduced a compositional state, specifically a heap (modelled as a function $\mathbb{N} \to Val$), which was required along with a more usual variable environment, to interpret any assertion. This heap or more generally resource, had an algebraic structure to it: the most fundamental one being a partial commutative monoid (PCM), with the binary operator dictating when it is valid to combine to heaps. Many more complex algebras for resources were studied in later works Calcagno et al. [2007]; Pym et al. [2004]. Separation logic was identified as a natural tool toreason about imperative programs mutating heap based memory at a suitable level of abstraction. It then found success also in reasoning about concurrent programs Brookes and O'Hearn [2016].

2.5.1 Reviewing Probabilistic Separation Logics

More recently there has been a lot of research into probabilistic separation logics: [Barthe et al., 2019], introduced separation as probabilistic independence. [Li et al., 2023] introduced Lilac, which allowed usage of continuous distributions, providing a measure theoretic definition of resources. [Ho et al., 2025] introduced reasoning about bayesian updates (supporting the `observe` and `normalize` constructs), thereby providing an axiomatic semantics for the general form of first order probabilistic programs (ie, BPPL as introduced by [Staton, 2017]).

There is a parallel line of work in relational separation logics. One could argue that relational logics are more important in the probabilistic domain, since in bayesian probabilistic programing (so including `observe` and `normalize`), frequently generates probability distributions which have no closed form solution! Sampling from these can only be done using approximation methods like Variational Inference. The justification for relational logics classical is that relational properties are expressed more naturally, but could perhaps still be done using unary reasoning, in an inelegant way. In probabilistic programs however, relational reasoning might in fact be more expressive making it important. Relational reasoning for probabilistic programs (though not using a separation logic) can be found here Barthe et al. [2009]. More recently there has been a novel combination of relational and unary reasoning for probabilistic programs in a separation logic by [Bao et al., 2025].

2.5.2 Lilac

The following are a few core intuitions behind Lilac. Lilac aims to reason about random variables, which is a higher level of abstraction that we generally reason in informally (as opposed to descending down into measure theory and lebesgue integrals). Since it's a separation logic, it is also compositional but along a different axis to compositionality of the denotational semantics. I call the compositionality of probabilistic separation logic a line-by-line compositionality (as opposed to term-wise compositionality). Each let binding creates a new "line" and we can compose our reasoning about each of line. In the denotational semantics we would instead reason compositionally about the arguments to the monadic bind operator (an s-finite measure and an s-finite-kernel). The former is a more natural way to reason about a sequencing operator.

This reasoning style is acheived by defining a resource which gets carried through and composed and split according to the laws of separation logic. This resource or state is a canonical source of randomness, the Hilbert cube, along with a measure space structure over it. It can be argued that the Hilbert cube could be replaced with something else, and indeed the definitions are parametric over this choice of measure space for a while. Any distribution can be constructed from the Hilbert cube (which is like a uniform distribution) using the operations/terms of the language.

`<insert: ownership as measurability>`

`<insert: independent combinations if probability spaces>`

2.6 Iris and MoSeL

Iris Jung et al. [2018] is a step-indexed modal separation logic partly designed as a framework for embedding other separation logics into, to take advantage of the Iris Proof Mode (IPM) Krebbers et al. [2017]. The Iris proof mode provides state of the art "tactical support", tactic based interactive theorem proving. More recently the front of the IPM was abstracted from Iris to any separation logic to create MoSeL Krebbers et al. [2018]. We aim to use MoSeL in this project for several reasons

1. Neither Lilac nor BaSL is step-indexed, so it is much cleaner to instantiate MoSeL rather than Iris
2. Making Lilac compatible with Lilac's interface (instantiating an algebraic structure called CMRA Jung et al. [2018]) is an unsolved problem (future work section Li et al. [2023])
3. MoSeL was completely ported to Lean, shortly before the start of this project! König [2022].

MoSeL presents the interface of a "logic of Bunched Implications" which essentially contains all standard connectives of a first order logic and the separation logic connectives. The only addition MoSeL adds to this is a persistence modality (which can be instantiated in a trivial way in exchange for defeating any automation it might achieve?).

One of the key generalisations that MoSeL makes over Iris is parametricity over linear and affine logics. Iris is an affine logic meaning separating conjuncts in a premise can be used 0 or 1 time in proving a goal; they can be thrown away if desired. In a linear logic however every conjunct must be used exactly once, as in the first separation logic by O'Hearn and Pym [1999]. There is naturally quite an emphasis on describing whether a logic is affine or linear, since throwing away unwanted predicates when it is suitable to do so, is exactly the kind of thing a user wants automation for. In BaSL Ho et al. [2025], there is a short discussion stating BaSL is part affine and part linear. Precisely characterising the constructs which are affine and linear or under what conditions should be explored for better tactical support.

MoSeL like IPM before it provides tactics mirroring the native Lean tactics, such as `iIntros` and `IAssumption`

In an ongoing project to port Iris from Rocq to Lean, MoSeL was completely ported to Lean right before the beginning of this project leanprover-community [2026]; König [2022]. So the formalisation builds on this. The interface provided by MoSeL is quite general. The interface defines a pre-orderered Set (PROP, Entails)

```
class BIBase (PROP : Type u) where
%   ⊢ : PROP → PROP → Prop -- Entails
  pure : Prop → PROP
  ∧ : PROP → PROP → PROP
  → : PROP → PROP → PROP
  * : PROP → PROP → PROP -- separating conjunct
  -* : PROP → PROP → PROP -- magic wand
  persistently : PROP → PROP
  ...
```

The type class `BIBase` defines the interface for the "data" parts of the separation logic, while the axioms that these need to satisfy (the "proof" parts) are defined in an extension of this type class. We are defining the separation logic, Lilac or BaSL, in the meta-logic, Lean. So `BIBase` is parametrised by a type `PROP` which is a proposition in the separation logic (as opposed to `Prop` in lean).

We draw the reader's attention to the `Entails` which allows us to provide the semantics of entailment. The soundness of MoSeL's syntactic entailments are established by the semantics provided when instantiating `Entails`. This gives us a little flexibility in how we'd like to instantiate `PROP`. For simpler logics we could have `PROP`, directly capture the semantis of that proposition, giving `PROP` this shape.

```
PROP := Store → Heap → Prop
```

My understanding is that this precisely encodes a relation: the satisfiability relation. An n-ary relation is encoded as a function from the n parameters to a Proposition. In other words it's a set of n-tuples. (Sets of type A are encoded as $A \to Prop$). Notice however that $s, h \vDash P$ should be of the form

```
PROP := Store → Heap → PROP → Prop
```

where did the P (of type `PROP`) go? It's given implicitly through the inductive construction of a term of `PROP` through the constructors defined by `and`, `sep`, etc. Any term of `PROP` implicitly contains the tree of constructors used to create it.

This is standard for the simple example instantiations of MoSeL (such as classical separation logic), but quite convoluted conceptuall. We may wish to split the representations of the separation logic assertions into syntax and semantics, which is what is done in non-trivial logics and this projects formalisation. There is a denotation function of type $Store \to Heap \to PROP \to Prop$ (exactly how the satisfiability relation should look) used in the instantiation of Entails; much more natural!

Chapter 3

Overview

3.1 Contributions

- We provide the first complete implementation of a probabilistic separation logic. Specifically Lilac is formalised in the Lean theorem prover complete with both soundness proof and extensive support for tactic-based weakest-precondition style reasoning about programs
- One of the first non-trivial instantiation of the Lean port of the Iris framework (originally developed in Rocq).
- A Novel treatment of infinite looping programs which may construct a convergent sequence of values. The correctness of some algorithms for probabilistic inference is given by convergence of such a sequence. So add a connective to Lilac for reasoning about infinite loops and re-establish the soundness of the logic.

  "The search for well-specified probabilistic models—models that correctly describe the processes that produce data—is one of the enduring ideals of the statistical sciences. Yet, in only the simplest of settings are we able to achieve this goal" - Papamakarios et al

  "Although our tactics perform symbolic execution in small steps, one can easily define a tactic that performs symbolic execution repeatedly. However, in concurrent separation logic, small step symbolic execution is often needed so as to be able to open invariants around atomic expressions" - Krebbers et al. 2017b https://iris-project.org/pdfs/2017-popl-proofmode-final.pdf

  so it makes sense for be to automatically apply wp_let, wp_ret and wp_unif and wp_ber immediately. However is it not possible for this to get stuck if we are not dealing with unif and ber alone (eg. complicate (det/rand)val coming from the meta logic??)

  Shall I talk about logic programming via type classes? Probably briefly should do. To illustrate why and how certain type classes were implemented for certain propositions. Especially interesting is this line [Krebbers et al. 2017b] "However, since we use a shallow embedding to represent the propositions of our object logic, propositions are semantic objects, so we cannot just write a Coq function to perform this traversal. Instead, we use logic programming using type classes to perform such manipulations on the meta level"

  This work is a good example exhibiting an easy to use separation logic proofmode, without the complexity of step-indexing (as in most iris projects). This is because our language is first-order and does not support recursion and yet has an orthogonal axis of intricacy which makes it interesting. As such it makes a good case study for the application of modern proofmode construction techniques decoupled from the step-indexed world of Iris in which practically every other significant program logic lives.

  Expressive tactic-based reasoning about programs is developed through Lean meta-programming.

3.2 Motivation

Here we give an informal account of what probabilistic programs are and why we would care to use them A Probabilistic programming language is not an established or general paradigm of programming such as Functional or Imperative programming. Most generally probabilistic programs are simply programs which have a `sample` construct for sampling from probability distributions, and optionally, an `observe` for specifying a new distribution given an old distribution and some newly observed data.

Statisticians use languages such as Stan Carpenter et al. [2017], Hakaru Narayanan et al. [2016] and Anglican Wood et al. [2014], to build complex statistical models succinctly using the expressivity of a programming language. Note that here, a model is simply a probability distribution over some quantity of interest, take for instance the probability of a 3d structure being correct for the given protein sequence, (other examples may come from weather forecasting, medical diagnosis, economics, statistical mechanics etc). "Running" this code corresponds to running a simulation of the model, where we can get answers such as the mean or variance, or just samples (sample protein structure to test in a lab in the example).

What distinguishes a probabilistic programming language is that it decouples the specification of a statistical model from the algorithm for its simulation (henceforth called probabilistic inference). Inference algorithms are often highly optimised and specialised to the specific model and the interpreter of a PPLs attempts to make the best choice/combination of inference algorithms for the given code (model), relieving the user of this additional work.

So in other words PPLs are a layer of abstraction which sits between a statistical model and the inference algorithms which simulate it. This abstraction unlocks the verification of correctness properties of the statistical model, which led to the development of program logics for reasoning about PPLs. This will be the main topic of this report.

3.2.1 Discrete vs Continuous distributions

First let us define what discrete and general (continuous) PPLs are. A discrete PPL does not allow specifying a probability distribution over $\mathbb{R}$. This is achieved most straightforwardly by not including a type corresponding to $\mathbb{R}$ in the syntax of a PPL. A general PPL will have a type corresponding to $\mathbb{R}$. This seemingly small detail has significant consequences on the semantics of a PPL which in turn dictates the complexity of program logics for such a language. This dichotomy largely exists due to difficulties with semantics of continuous PPLs. Take for example that a semantics for higher order PPLs was given by [Staton, 2017] within the decade.

However, all the PPLs above are continuous. Indeed PPLs are made only relevant when working with continuous distributions, as the remove the difficulty of working with the probabilistic inference algorithms. Much less sophistication is required in the inference algorithms for discrete algorithms, so discrete PPLs are not of much interest. So why are programs logics for discrete PPLs such as [Bao et al., 2025] and Lohse et al. [2026] important?

3.2.2 Probabilistic programs in the wild

In the most general sense a probabilistic program is just any program which samples from a distribution. Support for this can be found in the standard libraries of every major programming language, such as Python, C++ or Java. Some of critical applications manipulating discrete distributions, where formal verification is desirable is Cryptography and differential privacy . In fact one of the first major applications of probabilistic *separation* logic is being pursued in the formalisation of [Bao et al., 2025] to prove soundness of Zero Knowledge Proof Systems (subfield of Cryptography).

Chapter 4

Formalisation of Lilac

4.1 Syntax and semantics of Appl

To reason about Appl (see 2.3), we first need to define Appl in Lean. A first attempt at doing this may be inspired by how compilers are written: define an AST for the syntax of the language, and a typechecker for accepting or rejecting AST terms. Following such an approach would be painful given our goal of specifying properties of programs. We would always have to guard for the possibility that a program is not well-typed, in the assertion, and would lead to a proliferation of exception cases. Fortunately we are working in a dependently typed language, which is expressive enough to circumvent the problem by letting us define a syntax (AST) that is well-typed by construction.

For disambiguation, we refer to types in the *meta language* (in our case Lean) and *object language* (Appl), when it is unclear.

There are several standard ways to define object languages which are well-typed by construction, which all differ in the crucial problem of representing (object language) variables. This seemingly innocent problem of how to represent variables is one of the biggest issues encountered in this project, and standard techniques don't quite work for Appl, because of it's non-standard interpretation of (object language) types.

The final (Lean) representation of chosen for Appl follows a technique called Dependent De Brujin Indices [prefix Chlipala, 2013, see chap 17, Reasoning about programming language syntax]. Discussion on the alternatives are postponed to **??**.

To see why representing a programming language formally is nontrivial, and the difficulty

There is also an alternative

4.2 Pi-lambda-theorem as an induction principle

The π-λ-theorem is tool in measure theory which can be used for equality of two measures. We use it prove a certain uniqueness of measures theorem, used to establish that separating conjunction yeilds a PCM (partial commutative monoid) structure. It is similar to an induction principle on the structure of a σ-algebra. So we need prove some property C holds in the base and inductive cases of a σ-algebra. However there isn't an inductive type underlying this induction principle! The reason is two fold 1) when generating a measurable set from some base measurable sets, a specific set can be shown to be measurable by multiple different permutations of application of the axioms. For reference below is included an inductive defintion generating a σ-algebra from base measurable seats.

```
/-- The smallest σ-algebra containing a collection `s` of basic sets -/
inductive GenerateMeasurable (s : Set (Set α)) : Set α → Prop
  | protected basic : ∀ u ∈ s, GenerateMeasurable s u
  | protected empty : GenerateMeasurable s ∅
  | protected compl : ∀ t, GenerateMeasurable s t → GenerateMeasurable s tᶜ
```

```
| protected iUnion : ∀ f : ℕ → Set α, (∀ n, GenerateMeasurable s (f n)) →
    GenerateMeasurable s (⋃ i, f i)
```

2) The π-λ-theorem provides a relaxation on our proof obligation. We dont prove that the property C holds for countable unions (`iUnion` above) if it holds for each of the sets we're taking a union over. Instead we only need to do so for countably pairwise disjoint unions.

Being closed under disjoint unions instead of unions, is the difference between a λ-system compared to a σ-algebra. This relaxtion is the content of the π-λ-theorem which we don't go into here. However there is an alternative "inductive" definition of a λ system with the following axioms:

1. The system $\mathcal{C}$ containt the base measurable sets we chose
2. be closed under proper difference, for $\forall V, U \in \mathcal{C}, V \subseteq U \to U \setminus V \in \mathcal{C}$
3. closed under increasing limits, take $i \in \mathbb{N}$ and $U_i \in \mathcal{C}$ then $\bigcup_{i \in \mathbb{N}} U_i \in \mathcal{C}$

Unfortunately this is not the definition of λ-system in Mathlib, and it doesn't provide the "induction principle" according to this definition. For the case of probability measures (μ (universal set) = 1), there is a simple workaround, which is sufficient for Lilac. However BaSL uses unnormalised measures since it support bayesian conditioning, and so, there may be no simple workaround but reproving this alternative induction principle.

4.3 Binder Discipline

The choice between Dependent De Brujin Indices and Parametric Abstract Higher Order Syntax (PHOAS).

4.3.1 Dependent De Brujin Indices

- More straightforward to the uninitiated following a little more intuitively from the paper. Important if we want this is to be an accessible code base for future probabilistic separation logics to follow from.
- Measurability of maps expressible explicitly

4.3.2 PHOAS

- Notion of substitution inbuilt
- Example programs written in it are naturally readable. No separate parser and translation from a normal lambda calculus required. Correctness of the parser doesn't need to be proven... (but further consideration on this point requires deciding if there will be a separate executable implementation of the APPL syntax)

4.4 Equality of dependently typed records in Lean

4.5 Choice of Kripke Resource Monoid (KRM) over Unital ordered Resource algebra (ORA)

ORA's are established as the standard by former Rocq-iris projects, but I couldn't see why not just directly use the KRM. It was conceptual simplicity traded for some possibly unweildy proofs. The unweildiness turned out to be restricted to a single file?

4.6 Deep vs shallow embedding

4.7 Support for recursion over the data structure even with (shallow?/deep? see SSProve talk) embedding

(compare with SProp usage in Rocq for ORA? or something...) Even though the monotonicity property did not require induction, substitution atleast as defined in the paper requires it.

The aim, is to do things as given in the paper, and then think about if substitution could be done away with? If removing the ability to do recursion over assertions is desirable, etc

4.8 Strong abstractions across the proof development

4.8.1 The language

This is built on the existing abstractions. We use the typeclass `DenotationalMeas` to decouple the syntax from the language from the logic. The language only needs to me able to interpreted as measurable maps with a few additional requirements as given by `DenotationalMeas`.

4.8.2 Resource monoid

for the version of Lilac with/without conditioning

4.8.3 Separate treatment of BI connectives and Probability specific connectives

4.9 Metaprogramming and tactical support

Bibliography

Praveen Narayanan, Jacques Carette, Wren Romano, Chung-chieh Shan, and Robert Zinkov. Probabilistic inference by program transformation in hakaru (system description). In *International Symposium on Functional and Logic Programming - 13th International Symposium, FLOPS 2016, Kochi, Japan, March 4-6, 2016, Proceedings*, pages 62–79. Springer, 2016. doi: 10.1007/978-3-319-29604-3_5. URL http://dx.doi.org/10.1007/978-3-319-29604-3_5.

Bob Carpenter, Andrew Gelman, Matthew D. Hoffman, Daniel Lee, Ben Goodrich, Michael Betancourt, Marcus Brubaker, Jiqiang Guo, Peter Li, and Allen Riddell. Stan: A probabilistic programming language. *Journal of Statistical Software*, 76(1):1–32, 2017. doi: 10.18637/jss.v076.i01. URL https://www.jstatsoft.org/index.php/jss/article/view/v076i01.

Marco F. Cusumano-Towner, Feras A. Saad, Alexander K. Lew, and Vikash K. Mansinghka. Gen: a general-purpose probabilistic programming system with programmable inference. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2019, page 221–236, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450367127. doi: 10.1145/3314221.3314642. URL https://doi.org/10.1145/3314221.3314642.

Verified-zkEVM. Arklib: Formally verified arguments of knowledge in lean, 2025a. URL https://github.com/Verified-zkEVM/ArkLib. Accessed: 2025-12-15.

Joshua V. Dillon, Ian Langmore, Dustin Tran, Eugene Brevdo, Srinivas Vasudevan, Dave Moore, Brian Patton, Alex Alemi, Matt Hoffman, and Rif A. Saurous. Tensorflow distributions, 2017. URL https://arxiv.org/abs/1711.10604.

John M. Li, Amal Ahmed, and Steven Holtzen. Lilac: A modal separation logic for conditional probability. *Proc. ACM Program. Lang.*, 7(PLDI), June 2023. doi: 10.1145/3591226. URL https://doi.org/10.1145/3591226.

Shing Hin Ho, Nicolas Wu, and Azalea Raad. Bayesian separation logic, 2025. URL https://arxiv.org/abs/2507.15530.

Ralf Jung, Robbert Krebbers, Jacques-henri Jourdan, Aleš Bizjak, Lars Birkedal, and Derek Dreyer. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *Journal of Functional Programming*, 28:e20, 2018. doi: 10.1017/S0956796818000151.

Vladimir Gladshtein, George Pîrlea, Qiyuan Zhao, Vitaly Kurin, and Ilya Sergey. Foundational multi-modal program verifiers. *Proc. ACM Program. Lang.*, 10(POPL), January 2026. doi: 10.1145/3776719. URL https://doi.org/10.1145/3776719.

Sacha-Élie Ayoun, Opale Sjöstedt, and Azalea Raad. Soteria: Efficient symbolic execution as a functional library, 2025. URL https://arxiv.org/abs/2511.08729.

Verified-zkEVM. iris-lean. https://github.com/Verified-zkEVM/iris-lean, 2025b. Fork of the Iris Lean repository, instantiating a probabilistic separation logic, Bluebell.

Sam Staton. Commutative semantics for probabilistic programming. In *Programming Languages and Systems: 26th European Symposium on Programming, ESOP 2017, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2017, Uppsala, Sweden, April 22–29, 2017, Proceedings*, page 855–879, Berlin, Heidelberg, 2017. Springer-Verlag. ISBN 978-3-662-54433-4. doi: 10.1007/978-3-662-54434-1_32. URL https://doi.org/10.1007/978-3-662-54434-1_32.

Sheldon Axler. *Measures*, pages 13–72. Springer International Publishing, Cham, 2020. ISBN 978-3-030-33143-6. doi: 10.1007/978-3-030-33143-6_2. URL https://doi.org/10.1007/978-3-030-33143-6_2.

Michèle Giry. A categorical approach to probability theory. In B. Banaschewski, editor, *Categorical Aspects of Topology and Analysis*, pages 68–85, Berlin, Heidelberg, 1982. Springer Berlin Heidelberg. ISBN 978-3-540-39041-1.

Rémy Degenne. Markov kernels in mathlib's probability library, 2025. URL https://arxiv.org/abs/2510.04070.

Chris Heunen, Ohad Kammar, Sam Staton, and Hongseok Yang. A convenient category for higher-order probability theory. In *2017 32nd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, page 1–12. IEEE, June 2017. doi: 10.1109/lics.2017.8005137. URL http://dx.doi.org/10.1109/LICS.2017.8005137.

Michikazu Hirata, Yasuhiko Minamide, and Tetsuya Sato. Semantic Foundations of Higher-Order Probabilistic Programs in Isabelle/HOL. In Adam Naumowicz and René Thiemann, editors, *14th International Conference on Interactive Theorem Proving (ITP 2023)*, volume 268 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 18:1–18:18, Dagstuhl, Germany, 2023. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. ISBN 978-3-95977-284-6. doi: 10.4230/LIPIcs.ITP.2023.18. URL https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2023.18.

Reynald Affeldt, Cyril Cohen, and Ayumu Saito. Semantics of probabilistic programs using s-finite kernels in dependent type theory. *ACM Trans. Probab. Mach. Learn.*, 1(3), August 2025. doi: 10.1145/3732291. URL https://doi.org/10.1145/3732291.

Jeremy Avigad, Leonardo de Moura, Soonho Kong, and Sebastian Ullrich. *Theorem Proving in Lean 4*. 2024. URL https://leanprover.github.io/theorem_proving_in_lean4/. with contributions from the Lean Community.

Mario Carneiro. The type theory of lean. Master's thesis, Carnegie Mellon University, 2019. URL https://github.com/digama0/lean-type-theory/releases/download/v1.0/main.pdf.

Peter W. O'Hearn and David J. Pym. The logic of bunched implications. *Bulletin of Symbolic Logic*, 5(2):215–244, 1999. doi: 10.2307/421090.

Cristiano Calcagno, Peter W. O'Hearn, and Hongseok Yang. Local action and abstract separation logic. In *22nd Annual IEEE Symposium on Logic in Computer Science (LICS 2007)*, pages 366–378, 2007. doi: 10.1109/LICS.2007.30.

David J. Pym, Peter W. O'Hearn, and Hongseok Yang. Possible worlds and resources: the semantics of bi. *Theoretical Computer Science*, 315(1):257–305, 2004. ISSN 0304-3975. doi: https://doi.org/10.1016/j.tcs.2003.11.020. URL https://www.sciencedirect.com/science/article/pii/S0304397503006248. Mathematical Foundations of Programming Semantics.

Stephen Brookes and Peter W. O'Hearn. Concurrent separation logic. *ACM SIGLOG News*, 3 (3):47–65, August 2016. doi: 10.1145/2984450.2984457. URL https://doi.org/10.1145/2984450.2984457.

Gilles Barthe, Justin Hsu, and Kevin Liao. A probabilistic separation logic. *Proceedings of the ACM on Programming Languages*, 4(POPL):1–30, December 2019. ISSN 2475-1421. doi: 10.1145/3371123. URL http://dx.doi.org/10.1145/3371123.

Gilles Barthe, Benjamin Grégoire, and Santiago Zanella Béguelin. Formal certification of code-based cryptographic proofs. In *Proceedings of the 36th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '09, page 90–101, New York, NY, USA, 2009. Association for Computing Machinery. ISBN 9781605583792. doi: 10.1145/1480881.1480894. URL https://doi.org/10.1145/1480881.1480894.

Jialu Bao, Emanuele D'Osualdo, and Azadeh Farzan. Bluebell: An alliance of relational lifting

and independence for probabilistic reasoning. *Proc. ACM Program. Lang.*, 9(POPL), January 2025. doi: 10.1145/3704894. URL https://doi.org/10.1145/3704894.

Robbert Krebbers, Amin Timany, and Lars Birkedal. Interactive proofs in higher-order concurrent separation logic. *SIGPLAN Not.*, 52(1):205–217, January 2017. ISSN 0362-1340. doi: 10.1145/3093333.3009855. URL https://doi.org/10.1145/3093333.3009855.

Robbert Krebbers, Jacques-Henri Jourdan, Ralf Jung, Joseph Tassarotti, Jan-Oliver Kaiser, Amin Timany, Arthur Charguéraud, and Derek Dreyer. Mosel: a general, extensible modal framework for interactive proofs in separation logic. *Proc. ACM Program. Lang.*, 2(ICFP), July 2018. doi: 10.1145/3236772. URL https://doi.org/10.1145/3236772.

Lars König. An improved interface for interactive proofs in separation logic, October 2022.

leanprover-community. iris-lean: Lean 4 port of iris, a higher-order concurrent separation logic framework. https://github.com/leanprover-community/iris-lean/, 2026. Accessed: 18th Jan 2026.

Frank Wood, Jan Willem van de Meent, and Vikash Mansinghka. A new approach to probabilistic programming inference. In *Proceedings of the 17th International conference on Artificial Intelligence and Statistics*, pages 1024–1032, 2014.

Janine Lohse, Tim Rohde, Jimmy Xin, Niklas Mück, Iona Kuhn, Derek Dreyer, Deepak Garg, and Emanuele D'Osualdo. First steps towards probabilistic iris: Harmonizing independence, conditioning, and dynamic heap allocation, 2026. URL https://arxiv.org/abs/2605.13765.

Adam Chlipala. *Certified Programming with Dependent Types: A Pragmatic Introduction to the Coq Proof Assistant*. The MIT Press, 2013. ISBN 0262026651.